MAKERERE UNIVERISITY

FINANCE CREDIT SCORING PROJECT FINAL REPORT
Software Engineering
Group G

Name	Student Number	Registration number
Kalanzi Shahidah	2400705158	24/U/05158/PS
Kajunge Irene Namayanja	2400705098	24/U/05098/EVE
Mayanja Justus	2400706735	24/U/06735/PS
Kamwada Alex	2400722773	24/U/22773
Awongo fahadi Rashid	2400722713	24/U/22713

1 INTRODUCTION

For this project we built an end-to-end finance credit scoring system: starting from a raw, deliberately imperfect financial dataset, through data cleaning, feature engineering, exploratory data analysis, model training and evaluation, and finally a deployed web application that scores new applicants in real time.

Our dataset, finance_credit_scoring_dataset.csv , contains 1,480 records and 13 columns describing loan applicants and a target label, Risk_Level, with three classes: Low, Medium, and High. The columns are:

Column	Description
Customer_ID	Unique identifier for each applicant
Age	Applicant age in years
Annual_Income	Annual income (currency units)
Employment_Status	Salaried, Self-employed, Retired, Unemployed, or Student
Years_Employed	Years in current employment
Credit_History_Years	Length of credit history in years
Existing_Loans	Number of currently open loans

Debt_to_Income_Ratio	Debt-to-income ratio (%)
Credit_Utilization	Percentage of available credit currently used
Late_Payments	Count of late payments on record
Loan_Amount	Amount of the loan applied for
Credit_Score	Applicant credit score (standard 300-850 scale)
Risk_Level	Target: Low / Medium / High credit risk

The dataset was "unclean": it contained missing values, values written as words instead of numbers (e.g. "Thirty", "Seven Hundred", "25K USD"), out-of-range and negative values (e.g. ages below 0), and duplicated customer records. This made it a realistic exercise in data cleaning as well as modelling. All work is organised as a sequence of Jupyter notebooks in notebooks/ (InitialExploration.ipynb, ead.ipynb,cleaningdata.ipynb, feature_engineering.ipynb, training.ipynb, and the consolidated credit_score_final.ipynb), with cleaned data in data/processed/, saved models in models/, all charts in reports/figures/, and a Flask web application in app/ that we containerised with Docker and deployed at https://credit-scoring-ax3t.onrender.com/.

1.1 KEY OBJECTIVES

1. To source and understand a real-world credit risk dataset, identifying its structure, features, and target variable, so as to establish a reliable foundation for building a credit scoring pipeline.
2. To identify and correct data quality issues within the dataset including invalid entries, incorrect data types, and missing values in order to produce a clean, analysis-ready dataset.
3. To explore the relationships between customer characteristics and credit risk, and determine which features most strongly influence a client's likelihood of being classified as high, medium, or low risk.
4. To engineer meaningful new features from the existing data that may improve the model's ability to distinguish between risk categories.
5. To train and compare multiple machine learning models for predicting a client's credit risk level, and to evaluate which model performs best.

6. To investigate the impact of individual features on model predictions, particularly to determine whether features dominate the model's decisions, and whether this reflects genuine predictive value or data leakage from pre-computed scores.
7. To evaluate model performance using appropriate metrics.
8. To deploy the final trained model behind a simple, functional web interface, allowing a user to input customer details and receive a real-time credit risk prediction.
9. To assess the overall effectiveness and limitations of the pipeline, and to identify areas for potential improvement in future iterations.

2 EXPLORING THE DATASET AND MISSING DATA

Our first step was a structural audit of the raw. Basic facts:

- Shape: 1,480 rows x 13 columns.
- Several numeric-looking columns (Age, Loan_Amount, Credit_Score) were stored as text because a handful of rows used words instead of digits.
- df.duplicated() found 5 fully duplicated customer rows.
- df.describe() revealed impossible values once the text columns were coerced to numbers: a minimum Age of -10, a minimum Annual_Income of -45,000, a minimum Existing_Loans of -1, a minimum Credit_Utilization of -5%, a maximum Debt_to_Income_Ratio of 180.

Treating the disguised text values and the out-of-range/negative values as missing (rather than guessing at their "true" value) gave us the following missing-data picture, expressed as a percentage of the 1,475 rows remaining after duplicates were dropped:

Feature	Missing values	Percentage of rows
Credit_Score	3	0.20%
Age	2	0.14%
Annual_Income	2	0.14%
Loan_Amount	2	0.14%
Employment_Status	1	0.07%
Years_Employed	1	0.07%
Existing_Loans	1	0.07%
Credit_Utilization	1	0.07%

All other columns	0	0.00%

In total, only 13 cells out of roughly 19,175 data points (1,475 rows x 13 columns) were missing - about 0.07% of the entire dataset, and no single column exceeded 0.20%.

3 CHOOSING A METHOD FOR HANDLING MISSING DATA

We chose imputation, median for numeric columns, mode for the one categorical column with missing values (Employment_Status) for the following reasons:

- Missingness is extremely low (max 0.20% per column, 0.07% overall), far below thresholds (commonly 5-10%) where more elaborate approaches such as KNN or multiple imputation (MICE) would be justified.
- The missing values are scattered across different, unrelated rows rather than concentrated in one subgroup or correlated with the target, which is consistent with data missing completely at random (MCAR) rather than a systematic pattern we would need to model explicitly.
- We used the median rather than the mean because several numeric columns (Loan_Amount, Debt_to_Income_Ratio, Credit_Utilization) had visible outliers and skew even after removing the injected data-entry errors; the median is robust to that skew.
- Row deletion would have discarded otherwise complete, valid records.

4 IMPLEMENTING THE CLEANING/IMPUTATION AND EVALUATING ITS IMPACT

1. Converted text-disguised numbers back to numbers (e.g. "Thirty" -> 30, "Seven Hundred" -> 700, "25K USD" -> 25000) and coerced all numeric-like columns with pandas.to_numeric(errors="coerce")..
2. Flagged out-of-range and negative values as invalid and converted them to NaN (Age outside 18-100, Annual_Income < 0, Debt_to_Income_Ratio or Credit_Utilization outside 0-100, Credit_Score outside 300-850, negative counts).
3. Removed 5 exact duplicate customer records

4. Filled the remaining missing numeric values with each column's median,and the one missing Employment_Status with its mode ("Salaried").
5. Cast columns to their correct final data types (integers for counts, floats for ratios/currency, categorical for Employment_Status and Risk_Level) and saved the result to data/processed/cleaned_credit_risk_data.csv.

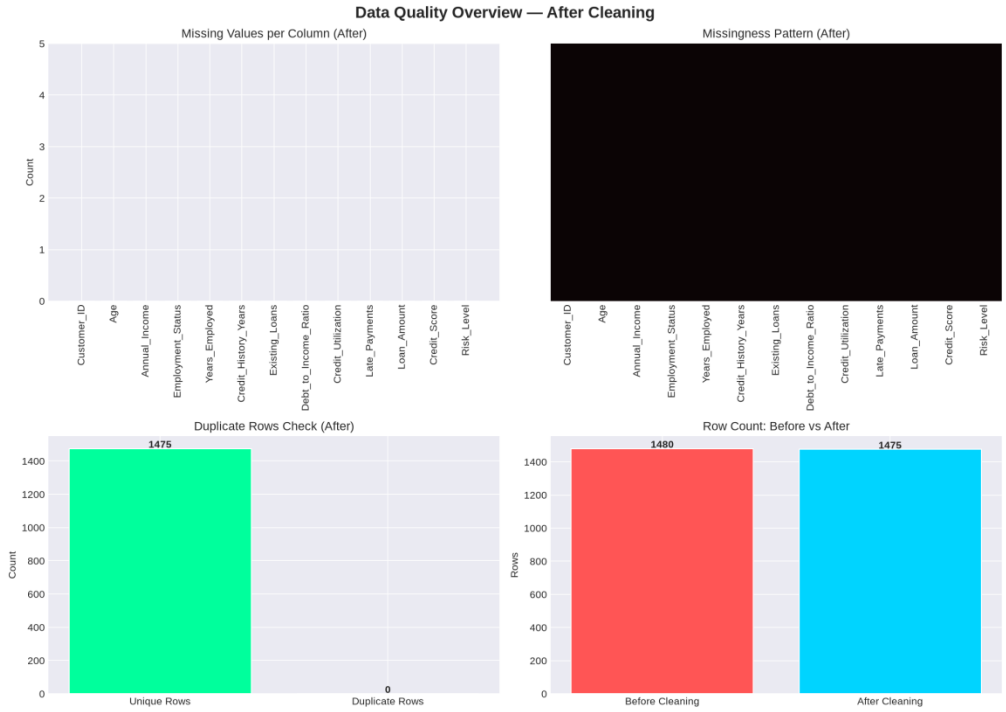


Figure 1Data quality overview after cleaning

Impact assessment: because the affected cells were so few, imputation barely moved the dataset's statistics. For example, Credit_Score's post-cleaning mean is 570.7 with a median of 568 - only 3 of 1,475 values (0.2%) were touched by imputation for that column.

5 EXPLORING THE DATASET FOR POTENTIAL FEATURES

With a clean dataset, we examined which variables actually relate to Risk_Level,

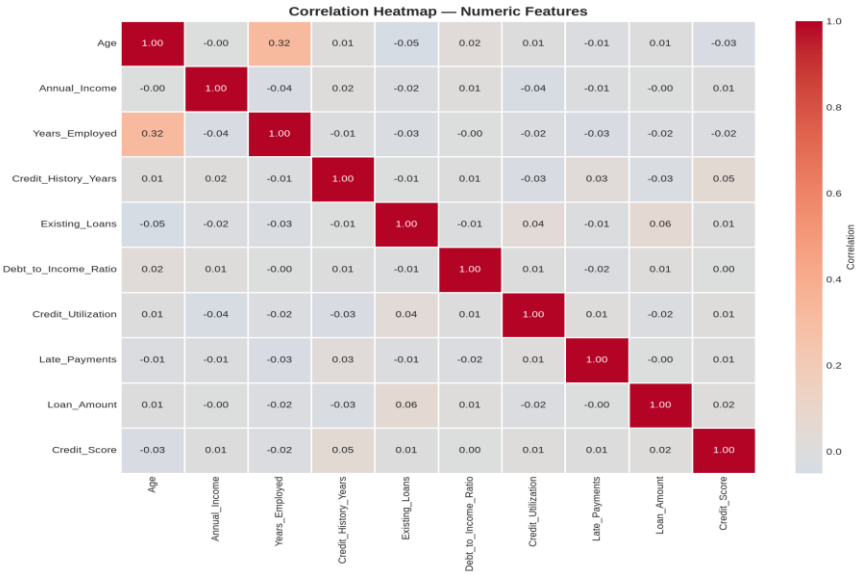


Figure 2Correlation heat map across the cleaned numeric features

Top predictors by absolute correlation with Risk_Level

Feature	Correlation with Risk_Level
Credit_Score	-0.89
credit_experience_ratio (engineered)	-0.07
Credit_History_Years	-0.07
Age	+0.04
Debt_to_Income_Ratio	-0.02
Loan_Amount	-0.02
Existing_Loans	-0.02
income_per_loan (engineered)	+0.01
Annual_Income	-0.01
employment_stability (engineered)	-0.01

The picture that emerged and which we return to throughout this report is that Credit_Score is overwhelmingly the strongest signal, roughly ten times more correlated with the target than any other single feature.

6 FEATURE ENGINEERING

We engineered the following features:

Ratio features (capture financial "shape" rather than raw magnitude):

- loan_to_income = Loan_Amount / Annual_Income - is the requested loan large relative to what the applicant earns?
- income_per_loan = Annual_Income / max(Existing_Loans, 1) - income available to service each existing loan.
- employment_stability = Years_Employed / max(Age - 17, 1) - proportion of an applicant's working life spent in their current job.
- credit_experience_ratio = Credit_History_Years / max(Age, 1) - proportion of an applicant's life spent building credit history.
- loans_per_year_employed = Existing_Loans / max(Years_Employed, 1) - rate of loan accumulation relative to work experience.

Binary risk flags (thresholds chosen from standard credit-risk practice):

- high_dti_flag: Debt_to_Income_Ratio > 40%.
- high_utilization_flag: Credit_Utilization > 70%.
- debt_burden_flag: either of the above two conditions.
- high_late_payment_flag: Late_Payments above the sample median.

Binned/categorical features:

- age_band: 18-30 / 31-45 / 46+.
- late_payment_band: Low (0-2) / Medium (3-5) / High (6+).
- credit_score_band: low_500 / 500_650 / 650_750 / 750_plus - used directly as a categorical feature (one-hot encoded) in the final production pipeline, alongside Employment_Status.

Together with the original 11 usable raw features (excluding Customer_ID), this produced a 20-feature table for the final production pipeline and a 27-feature, fully one-hot-encoded table for our exploratory model-comparison notebook.

7 EVALUATING THE IMPACT OF NEW FEATURES ON MODEL PERFORMANCE

Measuring "did feature engineering help" turned out to be more subtle than comparing two accuracy numbers, because Credit_Score alone is such a dominant predictor that it can mask the contribution of every other feature.

We therefore ran a two-part evaluation.

Part A - full feature set (raw + engineered) vs three models. WithCredit_Score included, Random Forest's feature importances show it capturing about 70% of total importance, with the next most useful features being almost entirely engineered ones:

Feature	Importance (Random Forest)
Credit_Score	0.70
credit_experience_ratio (engineered)	0.027
Debt_to_Income_Ratio	0.027
Credit_Utilization	0.024
loan_to_income (engineered)	0.022

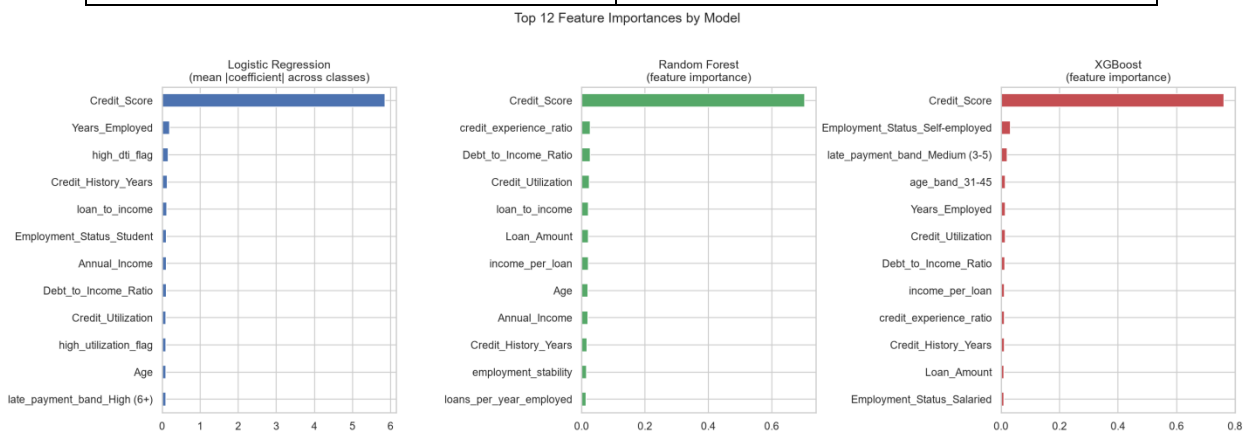


Figure 3Top-12 feature importances/coefficients for Logistic Regression, Random Forest, and XGBoost, side by side

Part B - ablation study : we retrained Random Forest and XGBoost with Credit_Score removed, to see what the engineered features and remaining raw features are worth on their own.

Model	Accuracy (with Credit_Score)	Accuracy (without Credit_Score)	F1-macro (without)
Random Forest	100.0%	47.5%	0.35
XGBoost	100.0%	52.2%	0.31

Interpretation: removing the one dominant field collapses performance from a perfect score to barely-better-than-random for a 3-class problem. This tells us two things at once:

- first, that our engineered do carry real, independent signal, performance is meaningfully above a naive baseline even without Credit_Score.

- second, and more importantly, that in the presence of Credit_Score they contribute only a small fraction of total predictive power, because Risk_Level in this dataset is overwhelmingly explained by Credit_Score alone.

8 VISUALIZATIONS, AND WHAT THEY TELL US

We used Matplotlib and Seaborn for the large majority of static charts, and Plotly for an interactive 3D exploration of the feature space.

8.1 Target and categorical distributions

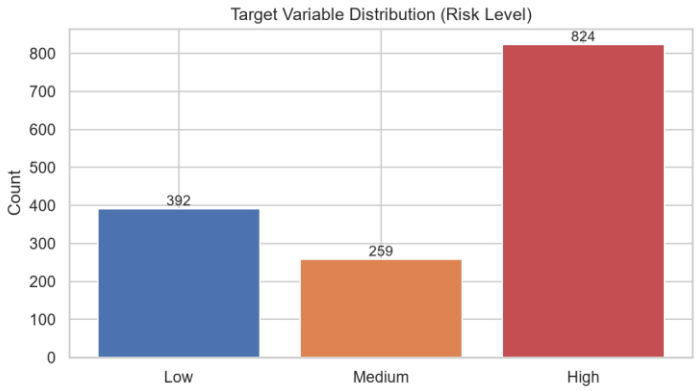


Figure 4: Class balance of Risk_Level

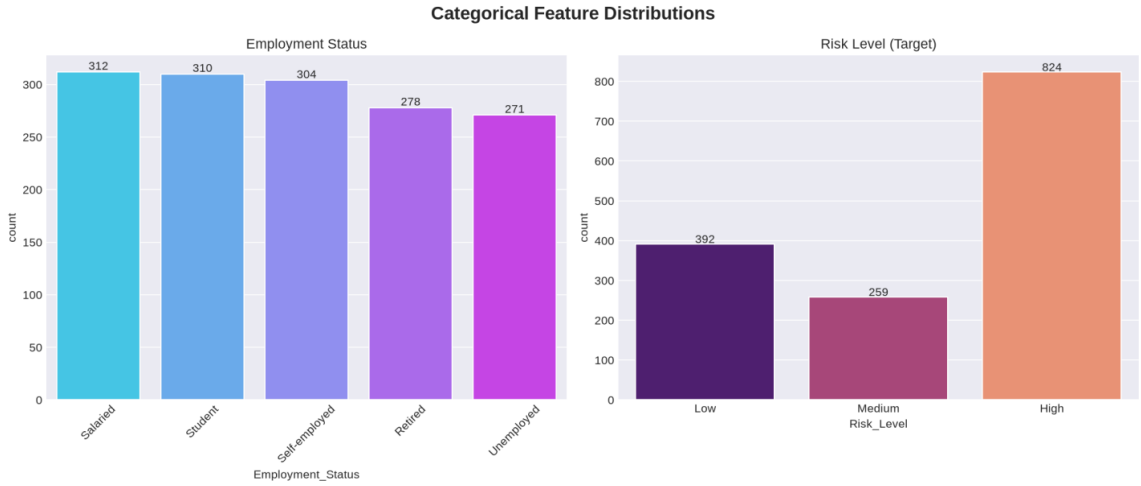


Figure 5Employment_Status counts (fairly balanced across five categories) alongside the Risk_Level target distribution

Interpretation: the target is moderately imbalanced (Medium is the minority class at under 18%), which is why we used macro-averaged metrics and class_weight="balanced" throughout modelling, rather than relying on plain accuracy.

8.2 Credit_Score - the standout variable



Figure 6Credit_Score plotted against Risk_Level per customer, checking for a clean numeric threshold between classes.

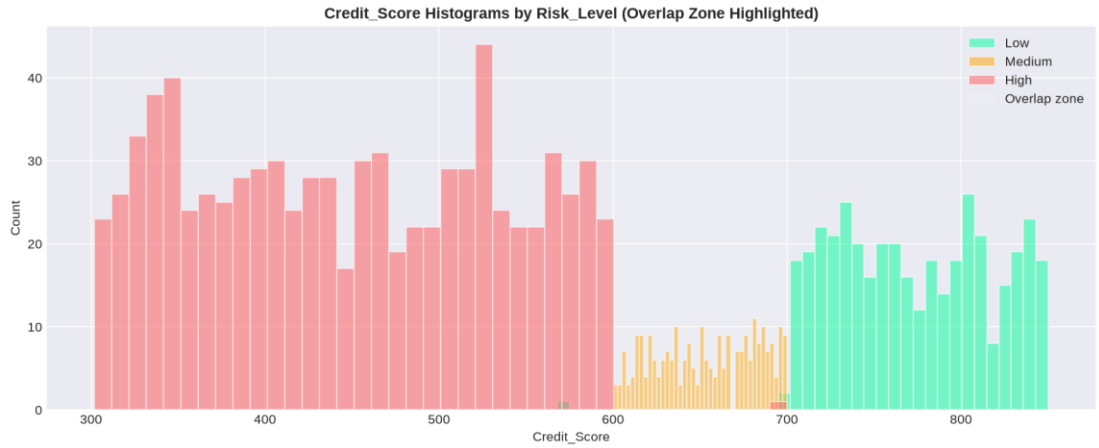


Figure 7Credit_Score histograms per Risk_Level, highlighting the 600-700 overlap zone

Interpretation: this is our most important visual finding. A simple rule, Credit_Score > 700 => Low, 600-700 => Medium, otherwise => High – correctly reproduces 1,473 of 1,475 labels (99.86%). The only ambiguity is a genuine overlap zone between 600 and 700, which contains 260 customers, 259 of them Medium and only 1 High. In other words, Risk_Level in this dataset behaves almost like a deterministic bucketing of Credit_Score rather than a label that independently reflects the other 20+ features we engineered.

8.3 Correlation, and top predictors

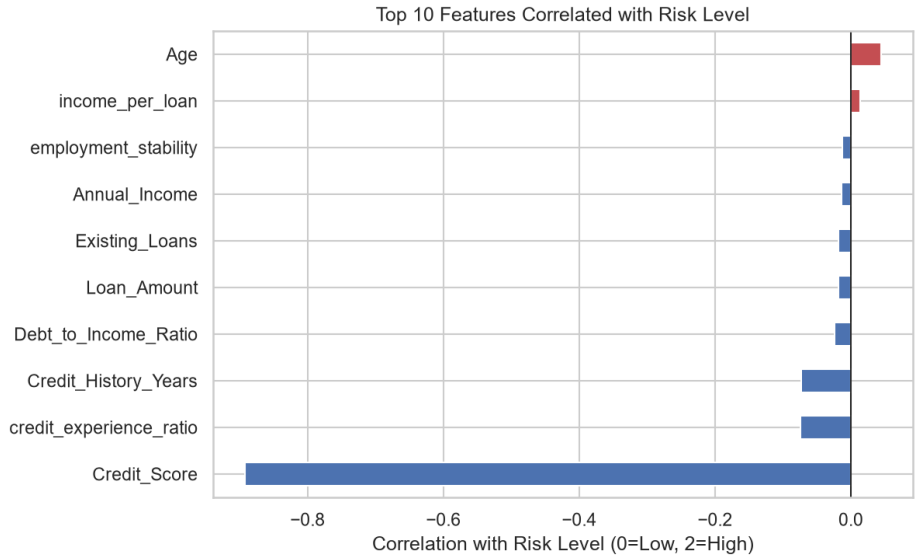


Figure 8Top-10 features ranked by absolute correlation with Risk_Level.

Interpretation: Employment_Status shows only mild differences (Self-employed applicants have the highest share of Low risk at 32%, Salaried the lowest at 22%), reinforcing that demographic/employment context plays a secondary role compared to Credit_Score.

8.4 A 3D, interactive view (Plotly)

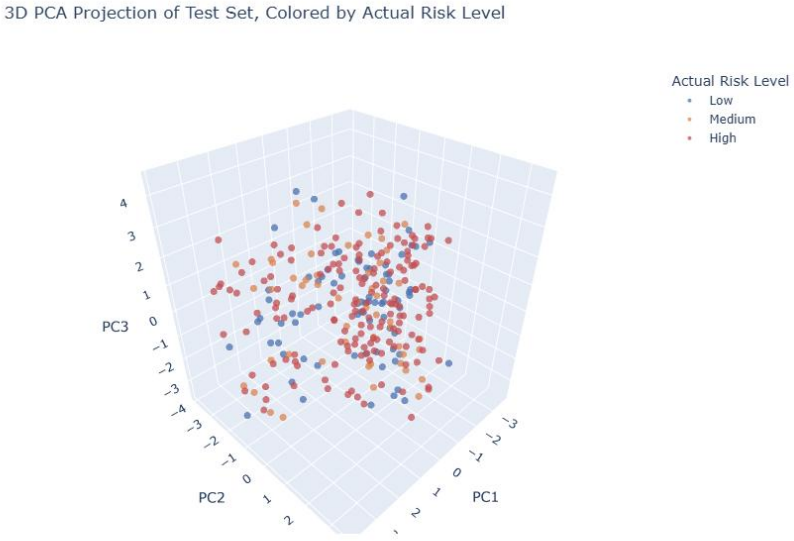


Figure 93D scatter of PC1, PC2, and PC3 (from Principal Component Analysis on the full feature set), coloured by actual Risk_Level.

Interpretation: even in a 3-dimensional PCA projection, the Low, Medium, and High classes form three visually distinguishable clusters with some touching/overlapping edges rather than three perfectly separated blobs - consistent with Logistic Regression's slightly-imperfect 95% accuracy versus the tree-based models' 100%, and with the 600-700 overlap zone identified above.

8.5 Model-evaluation visuals

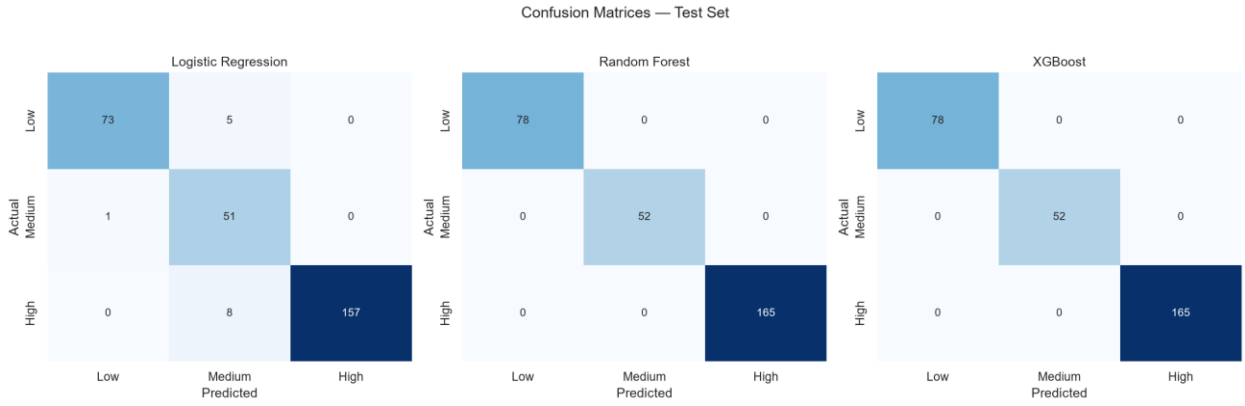


Figure 10: Confusion matrices on the held-out test set for Logistic Regression, Random Forest, and XGBoost.

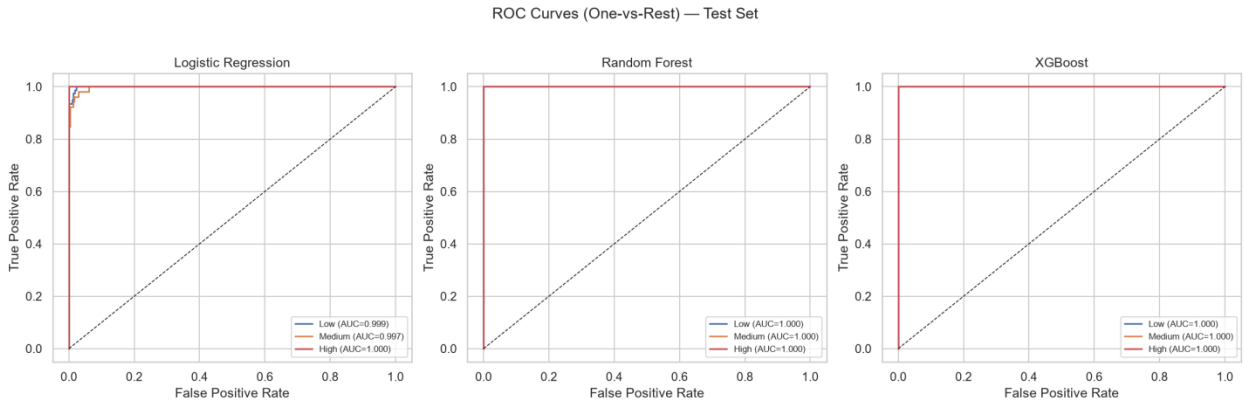


Figure 11One-vs-rest ROC curves per class, per model

9 DATA SPLITTING AND MODEL TRAINING

We used an 80/20 train/test split, stratified on Risk_Level so that both sets preserve the original class proportions (Low 26.6%, Medium 17.6%, High 55.9%):

Split	Rows	Low	Medium	High
Train	1,180	26.6%	17.5%	55.8%
Test	295	26.4%	17.6%	55.9%

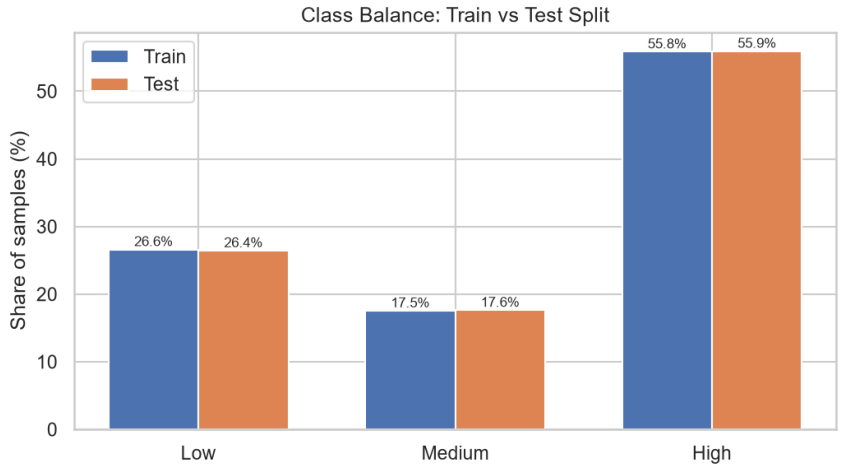


Figure 12Class balance confirmed to be near-identical between the train and test splits.

Preprocessing pipeline (scikit-learn ColumnTransformer, fit only on the training fold to avoid leakage): numeric features were median-imputed (as a safety net) and standardized (StandardScaler); the two categorical features (Employment_Status and credit_score_band) were one-hot encoded.

We trained and compared three classifiers, all wrapped in the same preprocessing pipeline:

- Logistic Regression (class_weight="balanced", C=1.0, max_iter=5000) – a simple, interpretable linear baseline.
- Random Forest (300-400 trees, class_weight="balanced") - a bagged ensemble that handles non-linearities and feature interactions.
- HistGradientBoostingClassifier (max_depth=6, early stopping) in our final notebook, and XGBoost (400 trees, max_depth=5, learning_rate=0.05) in our exploratory notebook - both modern gradient-boosting implementations, included to see whether boosting offered any edge over bagging.

10 CROSS-VALIDATION AND MODEL EVALUATION

We ran 5-fold stratified cross-validation on the training set (scoring: accuracy, precision/recall/F1-macro, ROC-AUC one-vs-rest weighted), then fit each model on the full training set and evaluated once on the untouched test set.

Cross-validation results (mean of 5 folds):

Model	CV Accuracy	CV F1-macro	CV ROC-AUC
Logistic Regression	95.1%	0.935	0.998
Random Forest	99.8%	0.998	1.000

HistGradientBoosting	99.6%	0.994	0.998

Held-out test-set results:

Model	Accuracy	Precision (macro)	Recall (macro)	F1-macro	ROC-AUC
Logistic Regression	93.9-95.3%	0.91-0.93	0.94-0.96	0.92-0.94	0.999
Random Forest	100.0%	1.00	1.00	1.00	1.00
HistGradientBoosting / XGBoost	100.0%	1.00	1.00	1.00	1.00

(The Logistic Regression range reflects two runs across our two notebooks with slightly different feature encodings; both land in the same 94-95% band.)

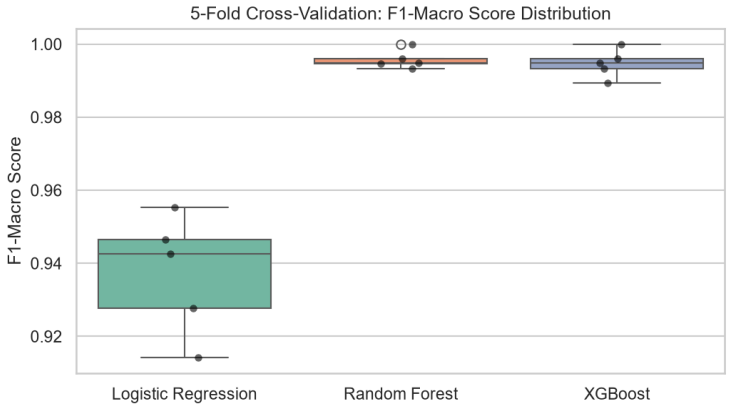


Figure 13Spread of F1-macro scores across the 5 cross-validation folds per model - Random Forest and the boosting models are tight and near 1.0; Logistic Regression is a little lower and more variable.

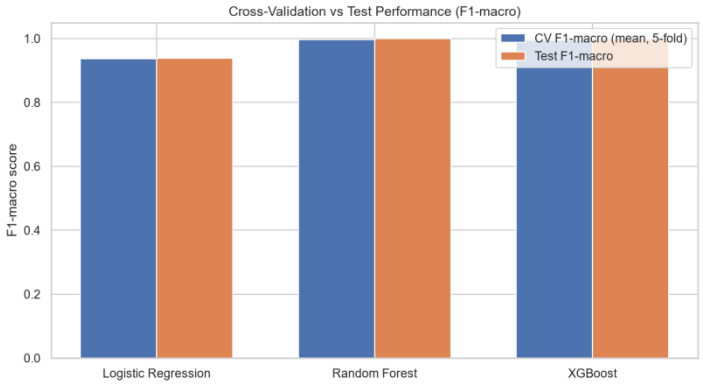


Figure 14: Cross-validation F1-macro vs test-set F1-macro per model – scores track closely, meaning the very high results are stable and not an artifact of one lucky train/test split.

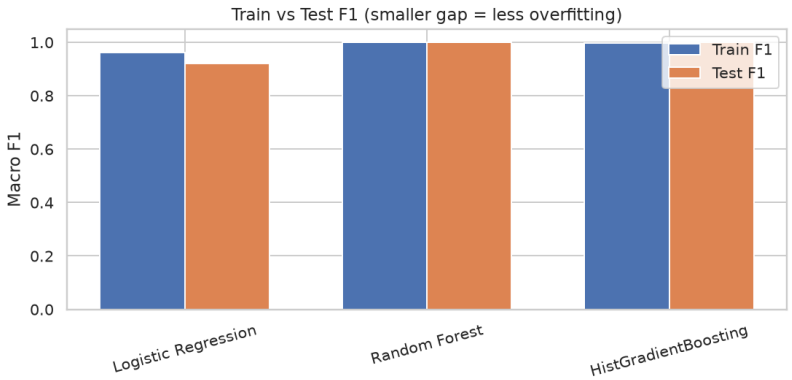


Figure 15Train F1 vs test F1 per model - all gaps are small (about 0.03-0.04 for Logistic Regression, near 0 for the tree ensembles), indicating no meaningful overfitting.

Model selection: we selected Random Forest as our final production model. It achieved the best combination of top-tier test performance (F1-macro = 1.00) and a negligible train/test gap (no overfitting), and it is more robust to the influence of any single unusual feature value than a single decision tree. This exact fitted pipeline (preprocessing + Random Forest) is saved as models/best_credit_risk_model.joblib and is the artifact loaded by the deployed web application.

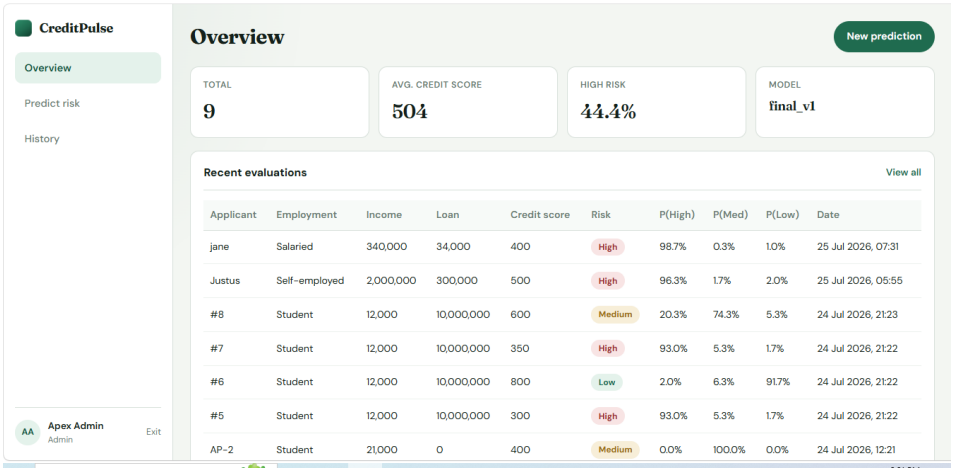
11 THE DEPLOYED WEB APPLICATION

To make the model usable beyond a notebook, we built a Flask web application ("CreditPulse") and deployed it with Docker on Render:

https://credit-scoring-ax3t.onrender.com/

Features:

- Authenticated access (Flask-Login) with an admin-seeded account.
- A prediction form covering all 11 applicant, with server-side validation
- Live scoring: the form submits to app/ml/service.py and returns the predicted risk label plus class probabilities.
- A dashboard summarising total applicants scored, the average credit score, and the share of High-risk predictions, plus the most recent predictions.
- A paginated history page of every prediction made through the app, persisted to a database via SQLAlchemy.



12 CONCLUSIONS, ACTIONABLE INSIGHTS, RECOMMENDATIONS, AND SUMMARY

Summary of what we did.

We audited a 1,480-row, 13-column credit applicant dataset, found and fixed data-quality issues affecting under 0.1% of all cells (missing values, disguised text, out-of-range and negative values, duplicate rows), engineered 9 additional features capturing debt burden, credit experience, and payment behaviour, explored the data with more than a dozen static and interactive visualisations, and trained/cross-validated three classification models on an 80/20 stratified split. We selected Random Forest as our final model (100% test accuracy, F1-macro = 1.00, negligible overfitting) and deployed it inside a Flask web application on Render.

Key deduction.

Risk_Level in this dataset is almost entirely determined by Credit_Score alone: a one-line threshold rule reproduces 99.86% of labels, and removing Credit_Score from the model collapses accuracy from 100% to 47-52% - barely bove chance for a 3-class problem. Every model, given Credit_Score, could solve this problem; the meaningful differences between models only show up once Credit_Score is removed.

Actionable insights and recommendations (future works):

- Treat Credit_Score as the primary decision input, but do not discard the next tier of variables. Once Credit_Score is excluded, credit_experience_ ratio, Debt_to_Income_Ratio, Credit_Utilization, and loan_to_income are the most

informative remaining features , they should still inform manual underwriting review, since they add independent signal beyond the score.

- For a more meaningful benchmark of model quality and of our feature engineering, we recommend evaluating the same pipeline on a dataset (or a redefined target) where Risk_Level is not a near-direct function of a single input column - for example, a real bank's realised default outcome rather than a pre-computed risk band.
- Logistic Regression's comparatively "imperfect" 93.9-95.3% accuracy is actually informative rather than a weakness: it shows the true decision boundary is not perfectly linear in Credit_Score, matching the overlap zone we found in the visualisations. It remains a good, fast, interpretable fallback model for the web application if a simpler or more explainable model is ever preferred over Random Forest.
- Operationally, the web application already gives us a production-ready path: every prediction is logged to a database (history page), which over time will let us monitor model drift and eventually validate predictions against real repayment .

Overall, this project delivered a complete pipeline: from a messy raw CSV,through justified and verified data cleaning, meaningful feature engineering, thorough visual and statistical exploration, rigorous model comparison with cross-validation, to a live, deployed web application.